

UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

- **Curso: Fundamentos de Programación**
- **Docente:** Américo Chulluncuy Reynoso

2026-1

Sesión 14:

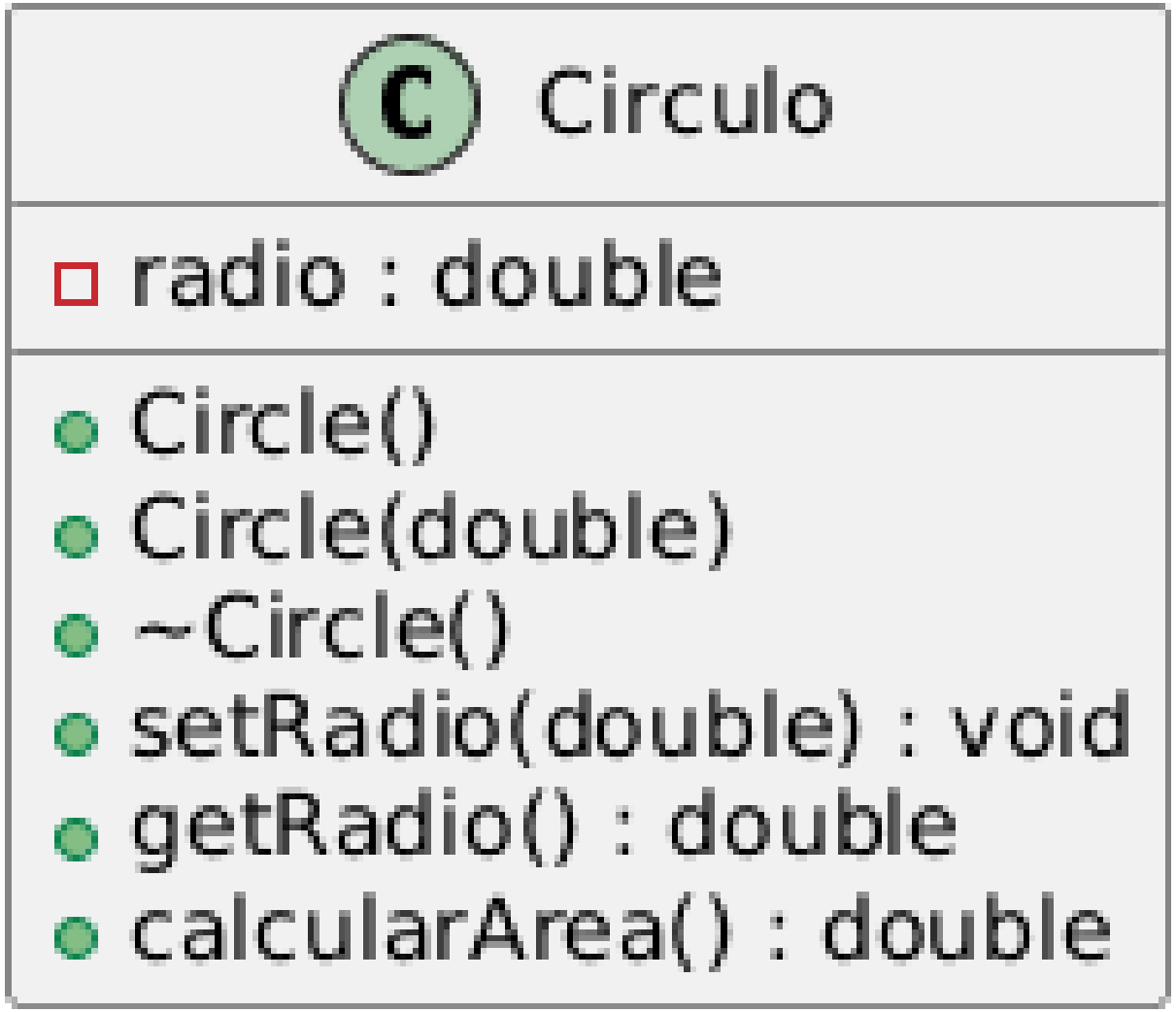
Los 4 pilares de la POO: Herencia y Polimorfismo

Contenido

1. Los 4 pilares de la POO
2. Herencia: la relación “es-un”
3. Implementando herencia en C++
   - Sintaxis básica
   - Constructores de la clase derivada
4. El problema: tratar objetos diferentes de forma uniforme
5. Polimorfismo: virtual, override, enlace dinámico
6. Diseño con polimorfismo
7. Síntesis y próximos pasos

¿Dónde estamos?

- **En la semana 13 construimos esto:**



// private

// constructor por defecto

// constructor con parámetros

// destructor

// valida invariante

Tenemos una clase Círculo que protege su estado, valida sus invariantes y gestiona su propio ciclo de vida. Hoy vamos a hacerla crecer

¿Y si ahora necesito un Cilindro?

Opción A:

```
// copiar y pegar
class Cilindro {
private:
    double radio;   // duplicado
    double altura;
public:
    Cilindro(double r, double h);
    double calcularArea(); // duplicado
    double calcularVolumen();
    // ...
};
```

- Si corriges un bug en Circulo, también debes corregirlo en Cilindro
- Si agregas validación en setRadio, debes recordar hacerlo en ambas
- Con 10 formas geométricas, tienes 10 copias del mismo código
- El compilador no sabe que Cilindro "es un" Circulo

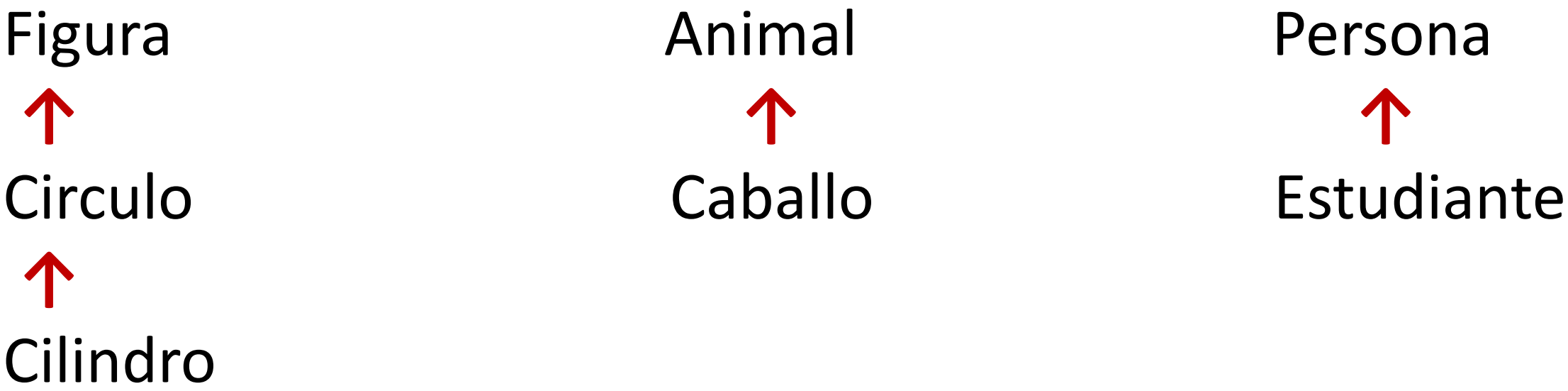
Necesitamos una forma de decirle al compilador: Cilindro **es un** Circulo con altura. Eso es lo que hace la **herencia**

1. Los 4 principios de la POO

Pilar	En nuestro código
Encapsulación (sesión 13)	`private radio` + getters/setters
Abstracción (sesión 13)	La interfaz `.h` oculta la implementación `.cpp`
Herencia (Hoy)	`Cilindro` extiende `Circulo`
Polimorfismo (Hoy)	Una función, comportamiento distinto según el objeto

La abstracción y la encapsulación **no son lo mismo**: la encapsulación protege los datos, la abstracción oculta la complejidad.

2. Herencia: derivar lo específico de lo general



- Hay una relación **es-un**: todo Cilindro es un Circulo. No al revés, es decir, Cilindro reutiliza todo lo que ya tenía Circulo y agrega algo más.

- La clase más general se llama **clase base**. La más específica: **clase derivada**.

¿Qué hereda la clase derivada?

SÍ se hereda:

- ✓ Atributos accesibles (public y protected)
- ✓ Funciones miembro (métodos)
- ✓ Interfaz publica de la clase base
- ✓ Relaciones de herencia de la clase base

NO se hereda (debe definirse):

- ✓ Constructores
- ✓ Constructor de copia
- ✓ Operador de asignación (=)
- ✓ Destructor propio de la derivada

Un Cilindro hereda el radio y los métodos del Círculo, pero sigue necesitando inicializar su propia altura.

Por eso los constructores no se heredan

El problema del `private:` en herencia

Private protege incluso de las clases derivadas. Necesitamos un nivel intermedio:

```
class Circulo {
private:
    double radio;   // Cilindro NO puede acceder a esto
};

class Cilindro : public Circulo {
public:
    double calcularVolumen() {
        return 3.14159 * radio * radio * altura; //  ERROR de compilación
    }
};
```

Una solución, usar:

```
class Circulo {
protected: // accesible para clases derivadas, no para el mundo
    double radio;
};
```

Tabla de acceso: ¿Quién puede ver qué?

Modifier	Access from same class	Access from subclass	Access from anywhere
private	Yes	No	No
protected	Yes	Yes	No
public	Yes	Yes	Yes

En la sesión 13 solo usamos private y public. Hoy incorporamos protected en algunos ejemplos por simplicidad.

Sin embargo en **diseños profesionales** suele preferirse private + métodos de acceso para mantener la encapsulación.

Sintaxis de herencia en C++

```
class Cilindro : public    Circulo {  };  // lo más común

class Cilindro : protected Circulo {  };

class Cilindro : private   Circulo {  }; // por defecto si
                                         // no se escribe nada
```

- Herencia pública (public): la más común, expresa una relación “es-un”, es la base del polimorfismo.
- Herencia protected y private: Usos más específicos, reutilización de implementación

3. Construyendo Cilindro paso a paso

Constructor de la clase derivada

El constructor derivado debe llamar al constructor base

EL PROBLEMA:

```
// ¿Quién inicializa radio?
Cilindro::Cilindro(double r, double h) {
    altura = h;
    // radio = r;  ERROR: radio es protected en Circulo,
    // pero el constructor de Circulo ya no corrió
}
```

LA SOLUCIÓN: lista de inicialización con llamada explícita

```
Cilindro::Cilindro(double r, double h) : Circulo(r), altura(h){
    cout << "Constructor Cilindro\n";
}
// primero se construye la parte Circulo
// luego la parte propia de Cilindro
```

REGLA: El constructor de la clase derivada **siempre** llama al de la clase base. Si no lo haces explícitamente, C++ llama al constructor por defecto de la base. Si la base no tiene constructor por defecto: error de compilación.

Orden de construcción y destrucción de Cilindro

```
Cilindro c(5.0, 10.0);
```

Ejecución:

```
  1. Constructor Circulo(5.0)    //  Constructor Circulo: r=5
  2. Constructor Cilindro(10.0) // Constructor Cilindro: h=10

// uso del objeto ...
```

Al salir del ámbito:

```
  3. Destructor ~Cilindro()  //  Destructor Cilindro
  4. Destructor ~Circulo()   //  Destructor Circulo
```

REGLA: Construcción: **base primero**, derivada después.
Destrucción: **derivada primero**, base después.

¿Compila o no compila?

Fragmento A

```
class Circulo {
protected:
    double radio;
public:
    Circulo(double r) : radio(r) {}
};


class Cilindro : public Circulo {
private:
    double altura;
public:
    Cilindro(double r, double h) : Circulo(r), altura(h) {}
    double getRadio() { return radio; }  //  ¿compila?
};
```

¿Compila o no compila?

Fragmento B

```
class Circulo {
private:              // cambio
    double radio;
public:
    Circulo(double r) : radio(r) {}
};


class Cilindro : public Circulo {
private:
    double altura;
public:
    Cilindro(double r, double h) : Circulo(r), altura(h) {}
    double getRadio() { return radio; }  // ← ¿compila ahora?
};
```

4. ¿Cómo calcular el área de varias figuras?

Si tengo 2 figuras

```
Circulo c(3);
Cilindro ci(3,5);
cout << c.calcularArea() << endl;
cout << ci.calcularArea() << endl;
```

Si tengo muchas figuras

```
Circulo c1(3);
Circulo c2(4);
Rectangulo r1(2,5);
Cilindro ci1(3,5);
Esfera e1(2);
```

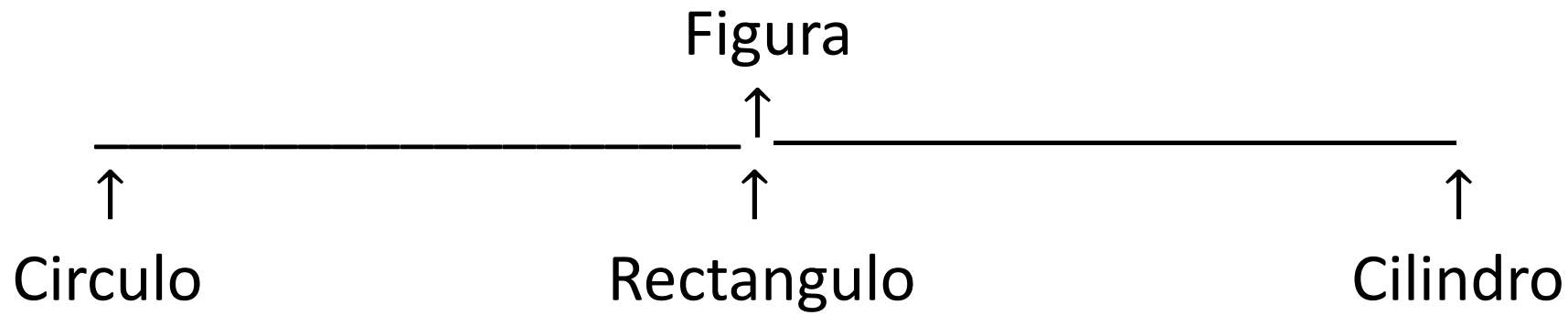
```
c1.calcularArea();
c2.calcularArea();
r1.calcularArea();
ci1.calcularArea();
e1.calcularArea();
```

Si queremos agregar otra figura?

Nuestro programa funciona hoy. El problema es que no está preparado para crecer. Cada vez que agregamos una nueva figura debemos volver a modificar el código que calcula las áreas.

¿No sería mejor que el algoritmo trabajara con cualquier figura, incluso con clases que todavía no existen hoy?

Quiero tratar todas las figuras de la misma manera



¿Qué tienen en común todas?

calcularArea()

```
class Figura {
public:
    double calcularArea();
};
```

Lo que nos gustaría escribir

```
Figura* formas[3];
// es legal porque existe herencia
formas[0] = &circulo;
formas[1] = &rectangulo;
formas[2] = &cilindro;

for(int i=0;i<3;i++) {
    formas[i]->calcularArea();
    // ¿cuál versión ejecuta?
}
```

¿Qué debería imprimirse?

```
Area del círculo
Area del rectángulo
Area del cilindro
```

El problema inesperado

```
class Figura {
public:
    double calcularArea() {
        return 0.0;
    }
};

Figura* f = &circulo;
cout << f->calcularArea();
```

Resultado

0

¿Por qué falla?

```
class Figura {
public:
    double calcularArea() { return 0.0; }  // sin virtual
};

class Circulo : public Figura {
public:
    double calcularArea() { return 3.14159 * radio * radio; }
};
```

```
// RESULTADO
Figura* f = new Circulo(3.0); // Es legal porque existe herencia public (upcasting)
cout << f->calcularArea();   // imprime 0.0 – ejecuta Figura::calcularArea
```

Figura *f El compilador ve: Figura. No ve el tipo del objeto real: Circulo.

Entonces ejecuta: Figura::carlcularArea(). Esto es **enlace estático.**

5. Solución: virtual y enlace dinámico

```
class Figura {
public:
    virtual double calcularArea() { return 0.0; }
};

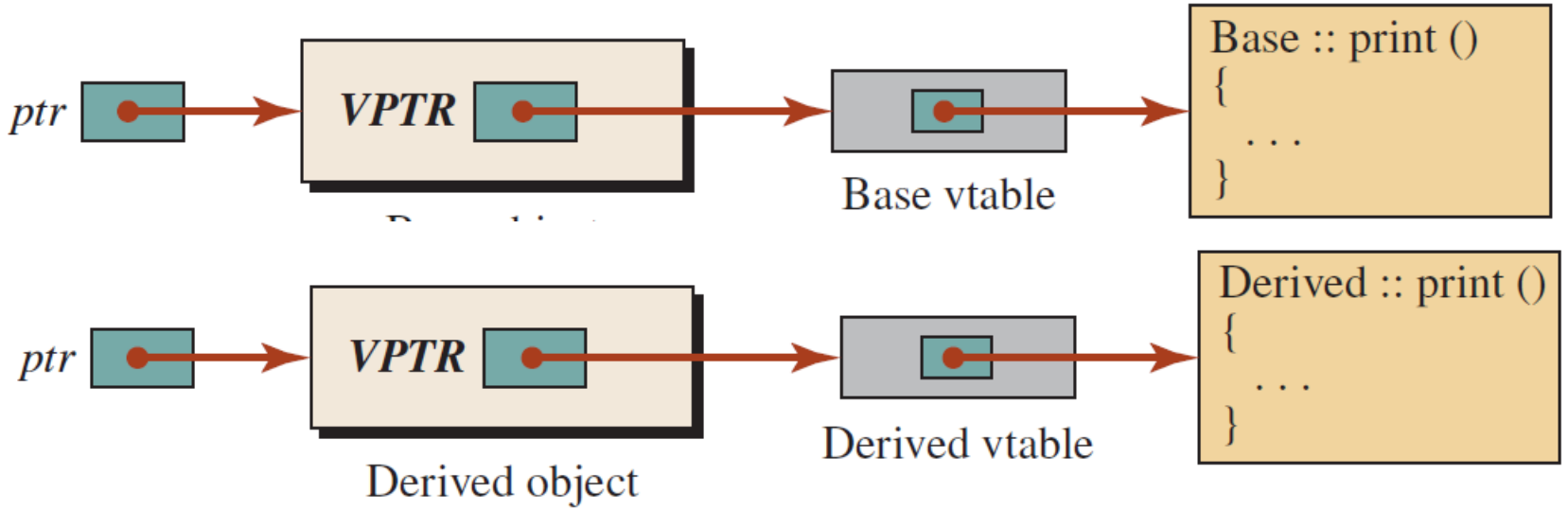
class Circulo : public Figura {
public:
    double calcularArea() override { return 3.14159 * radio * radio; }
                        // buena práctica: indica que sobreescribe
};

class Cilindro : public Circulo {
public:
    double calcularArea() override {  }
};

Figura* f = new Circulo(3.0);
cout << f->calcularArea();   // ahora imprime 28.27431  ¡CORRECTO!
```

Polimorfismo dinámico: la versión de la función que se ejecuta se decide en tiempo de ejecución, según el tipo real del objeto, no el tipo del puntero.

Mecanismo interno: vtable y VPTR



- Cada clase con funciones virtual genera una vtable (tabla de punteros a funciones)
- Cada objeto contiene un VPTR oculto que apunta a la vtable de su clase real
- Al llamar ptr->calcularArea(); C++ sigue el VPTR → vtable → función correcta
  Figura*ptr -> [objeto Circulo]
  VPTR -----> Circulo vtable ---> calcularArea ---> Circulo::carlcularArea()
  radio

Los 3 ingredientes del Polimorfismo Dinámico

- **Herencia** (familia de clases)

  +
- **Puntero o referencia a la clase base** (visión común)

  +
- **Método virtual** (comportamiento variable)

  =

La llamada se resuelve según el tipo real del objeto (en tiempo de ejecución)

Si falta cualquiera de los tres, la llamada se resuelve en tiempo de compilación

En términos simples, el mismo mensaje (calcular Area()) puede producir comportamientos distintos según el objeto que lo recibe.

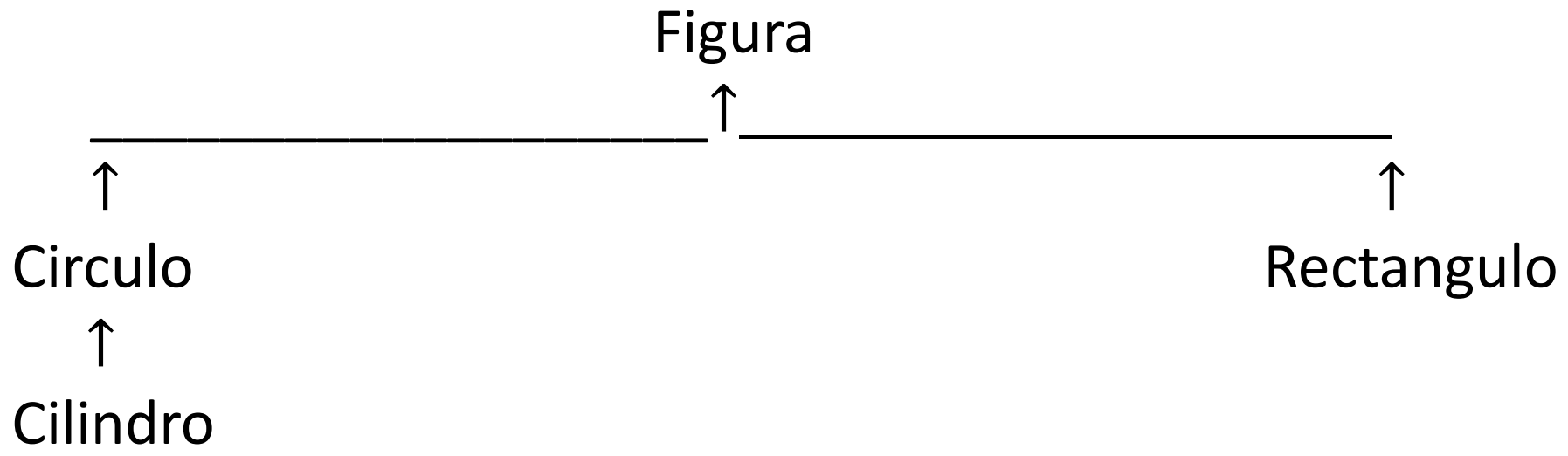
¿Qué imprime este código?

```
class Figura {
public:
    void imprimir()             { cout << "Figura\n"; }
    virtual void describir()    { cout << "Soy una figura\n"; }
};

class Circulo : public Figura {
public:
    void imprimir() { cout << "Circulo\n"; }
    void describir() override { cout << "Soy un circulo\n"; }
};


int main() {
    Figura* f = new Circulo();
    f->imprimir();   //
    f->describir();  //
}
```

6. Diseño con polimorfismo




```
Figura* formas[3];
formas[0] = new Circulo(3.0);
formas[1] = new Cilindro(3.0, 5.0);
formas[2] = new Circulo(7.0);


for (int i = 0; i < 3; i++) {
    cout << "Area: " << formas[i]->calcularArea() << "\n";
}
// Cada llamada ejecuta la versión correcta automáticamente
```

Si mañana agregamos una nueva figura, solo incorporamos un nuevo objeto a la colección. El algoritmo que procesa las figuras **permanece exactamente igual**.

virtual vs override: buenas prácticas

Palabra	Dónde va	Qué hace
`virtual`	En la clase **base**	Declara que la función puede sobreescribirse
`override`	En la clase **derivada**	Confirma que estás sobreescribiendo una función virtual (el compilador verifica)

```
class Figura {
public:
    virtual double calcularArea() const;
};

class Circulo : public Figura {
public:
    // sin override el compilador no avisa si cometes este error:
    double calcularArea();   // falta const, NO sobreescribe, crea función nueva

    // con override el compilador sí avisa:
    double calcularArea() override;   // ERROR: firma no coincide con la base
};
```

Introducción a clases abstractas

- ¿Tiene sentido crear un objeto Figura?

```
Figura f; // ¿qué área tiene una "figura" genérica? No tiene sentido.
```

```
class Figura {
public:
    virtual double calcularArea() const = 0;  // función virtual pura
};
// Figura ahora es una clase abstracta: no se puede instanciar directamente
// Solo sirve como contrato para las clases derivadas
```

```
Figura f;         // ERROR: no se puede instanciar clase abstracta
Circulo c(3.0);   // OK: implementa calcularArea()
Figura* p = &c;   // OK: puntero a clase abstracta sí puede usarse
```

7. Lo que construimos

PROBLEMA ORIGINAL

`struct Circulo + funciones sueltas → radio = -5 posible`

↓

ENCAPSULACIÓN (S13)

`class Circulo { private: double radio; }`

↓

ABSTRACCIÓN (S13)

Interfaz circulo.h oculta implementación circulo.cpp

↓

HERENCIA (S14)

`class Cilindro : public Circulo`

→ reutilización, jerarquía, protected

↓

POLIMORFISMO (S14)

`virtual calcularArea()`

→ un puntero Figura*, muchos comportamientos

Lo que la clase Circulo ya cumple

Sesión 13:

- ✓ Encapsula `radio` como `private`
- ✓ Valida invariantes en constructor y setter
- ✓ Constructor por defecto, con parámetros, destructor
- ✓ Getter `const`, setter con validación
- ✓ Separación `.h` / `.cpp`

Sesión 14:

- ✓ Sirve como clase base para `Cilindro`
- ✓ Expone `radio` como `protected` para clases derivadas
- ✓ Declara `calcularArea` como `virtual`
- ✓ Puede usarse polimórficamente a través de `Figura*`

Lo que viene después

- **POO (Java / C++ avanzado)**
- **Estructuras de Datos (Listas, Pilas, Colas, Árboles, Grafos)**
- **Diseño de Algoritmos**
- **Ingeniería de Software**

Más adelante descubrirán que conceptos como:
Clases abstractas
Herencia múltiple
Plantillas (templates)
STL (vector, map, algorithm)
Se construyen sobre las mismas ideas que estudiamos en el curso